

Documentação Técnica – Tema 7

Gerador Paralelo de Testes e Análise de Código

Disciplina: Ciência da Computação – Programação Distribuída e Paralela
Integrantes: Gustavo Yuji Virgolino Nishimura, Heitor Yasuo Yamamoto
Professor: Fábio Rocha

Declaração de uso de IA: Neste projeto foi utilizado a IA Gemini, para esclarecimento de dúvidas sobre o projeto e geração do código base, passando pela análise e aprendizado da equipe. Também utilizamos para configurar os serviços de EC2 , SQS e S3 na AWS. A Claude IA teve participação na revisão e criação do código e na montagem da documentação técnica com os conteúdos devidos.

1. Introdução

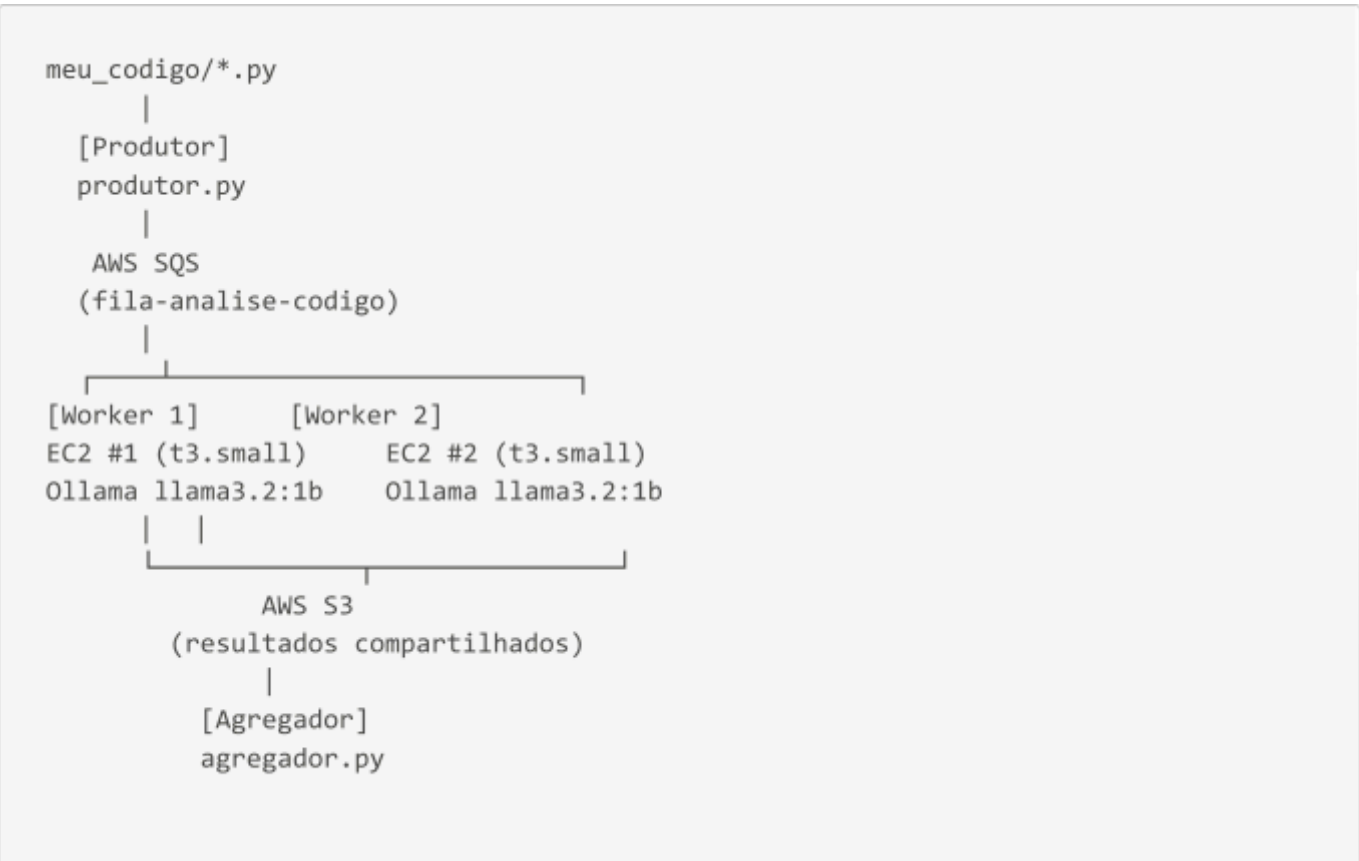
Este documento descreve a arquitetura, decisões técnicas e análise experimental de um sistema distribuído para geração paralela de testes unitários e análise de código Python. O sistema recebe arquivos de código-fonte, distribui o processamento entre múltiplos workers via fila de mensagens AWS SQS, e utiliza um Small Language Model (SLM) auto-hospedado para gerar os testes e identificar problemas de qualidade no código.

A abordagem adotada é o **Tema 7 – Gerador Paralelo de Testes e Análise de Código**, explorando os conceitos de paralelismo embaraçoso (embarrassingly parallel), agregação distribuída de resultados e detecção de conflitos.

2. Arquitetura do Sistema

2.1 Visão Geral

O sistema é composto por quatro componentes principais que se comunicam de forma assíncrona, distribuídos em duas instâncias EC2 independentes:



2.2 Componentes

Componente	Responsabilidade
produtor.py	Escaneia meu_codigo/, serializa cada arquivo e envia como mensagem SQS
worker.py	Consume mensagens da fila, chama o LLM e salva resultados no S3
agregador.py	Lê resultados do S3, consolida métricas e detecta conflitos
utils.py	Retry com backoff e logging estruturado compartilhados
Ollama + llama3.2:1b	Servidor de inferência SLM em cada instância EC2

2.3 Infraestrutura AWS

Serviço	Uso
EC2 (2x t3.small)	Uma instância por worker, cada uma com Ollama próprio
SQS	Fila principal de análise (fila-analise-codigo)
SQS (DLQ)	Dead Letter Queue para mensagens com falha reiterada
S3	Storage compartilhado de resultados entre instâncias

3. Justificativa das Escolhas de Paralelismo e Distribuição

3.1 Paralelismo Embaraçoso

A análise de arquivos Python independentes é um exemplo clássico de **paralelismo embarçoso**: cada arquivo pode ser processado de forma completamente independente, sem dependência de dados entre as tarefas. Isso elimina a necessidade de sincronização durante o processamento e maximiza o potencial de paralelismo.

3.2 Modelo Produtor-Consumidor via SQS

A escolha do AWS SQS como mecanismo de comunicação se justifica por:

- **Desacoplamento**: o produtor não precisa conhecer quantos workers existem nem seu estado
- **Balanceamento de carga automático**: o SQS distribui mensagens entre workers disponíveis
- **Tolerância a falhas nativa**: mensagens não processadas retornam à fila após o **VisibilityTimeout** (120s)
- **Escalabilidade**: novos workers podem ser adicionados sem alteração no código

3.3 S3 como Storage Compartilhado

Com workers em instâncias EC2 separadas, cada um possui disco local independente. O S3 resolve o problema de agregação distribuída: todos os workers escrevem resultados no mesmo bucket, e o agregador

lê de lá — independentemente de qual instância processou cada arquivo.

3.4 SLM Auto-Hospedado vs API Gerenciada

O projeto utiliza **Ollama com llama3.2:1b** em vez do Amazon Bedrock por:

1. **Disponibilidade:** a conta AWS Academy de ambos os integrantes não oferece acesso ao Bedrock
2. **Controle total:** permite observar o impacto do paralelismo diretamente no throughput
3. **Bônus pedagógico:** demonstra o paralelismo interno do servidor de inferência

4. Harness Engineering e Engenharia de Contexto

4.1 O Harness

O `worker.py` é o **harness** do sistema — a estrutura que orquestra todas as chamadas ao LLM com contexto controlado:

```
worker.py ← HARNESS
|
├──
├── carregar_system_prompt() → carrega contexto versionado do arquivo
├── chunkar_codigo() → gerencia janela de contexto do LLM
├── chamar_ia() → chama modelo com system + user prompt
├── combinar_testes() → pós-processa saída de múltiplos chunks
├── chamar_com_retry() → tolerância a falhas nas chamadas
├──
└── MODO_ANALISE → troca o comportamento sem alterar código
```

4.2 System Prompts Versionados

Os prompts são mantidos em arquivos separados do código, no diretório `prompts/`, permitindo evolução independente sem tocar na lógica de processamento.

v1 — Geração de Testes (`prompts/v1_gerar_testes.txt`):

Define o modelo como especialista em QA com PyTest. Impõe regras estritas de formato: apenas código Python puro, sem blocos markdown, cobrindo casos normais, de borda e de erro. Isso garante que a saída seja executável diretamente pelo `pytest`.

v2 — Detecção de Code Smells (`prompts/v2_detectar_smells.txt`):

Define um schema JSON de saída estruturado com campos padronizados (`tipo`, `linha`, `descricao`, `sugestao`, `nota_qualidade`, `resumo`), garantindo consistência entre diferentes workers e arquivos.

4.3 Estratégia de Chunking

Para arquivos que excedem 3.000 caracteres, o worker divide o código nos pontos onde uma nova função (`def`) ou classe (`class`) começa — preservando unidades semânticas completas. Após processar múltiplos chunks, `combinar_testes()` elimina imports duplicados e une os corpos dos testes em um único arquivo coerente.

4.4 Seleção do Modo via Variável de Ambiente

O modo de análise é selecionado via `MODO_ANALISE`, permitindo alternar entre geração de testes e detecção de code smells sem modificar o código:

```
MODO_ANALISE=smells python3 worker.py
```

4.5 Pós-processamento da Saída

O worker garante qualidade da saída do LLM:

- Remove blocos de código markdown (```) que o modelo eventualmente inclui
- Verifica se o import obrigatório (`from meu_codigo.<modulo> import *`) está presente e o adiciona caso ausente

5. Métricas e Observabilidade

5.1 Logs Estruturados

Cada componente gera logs em formato JSON com timestamp, nível, identificador e mensagem. Workers têm ID único (`WORKER_ID`) que permite rastrear qual instância processou cada arquivo:

```
{"ts": "2026-05-26T14:23:01", "level": "INFO", "worker": "7c5b2961", "modo": "testes", "msg": "Processando: calculadora.py | 32 chars"}
```

5.2 Métricas por Arquivo

Métrica	Descrição
<code>latencia_segundos</code>	Tempo de processamento individual
<code>tokens_prompt</code>	Tokens enviados ao modelo
<code>tokens_resposta</code>	Tokens gerados pelo modelo
<code>chunks_processados</code>	Número de partições do código
<code>worker_id</code>	Instância responsável pelo processamento
<code>status</code>	Sucesso ou falha

5.3 Relatório Consolidado

O agregador calcula as métricas globais ao final:

- **Tempo CPU acumulado:** soma das latências de todos os workers em todas as instâncias
- **Tempo real de relógio:** tempo total desde o início até a conclusão
- **Throughput:** arquivos processados por segundo

- **Detecção de conflitos:** identifica módulos processados mais de uma vez

6. Tolerância a Falhas

6.1 Retry com Exponential Backoff

Todas as chamadas a serviços externos são protegidas por retry com backoff exponencial e jitter aleatório (implementado em `utils.py`):

```
Tentativa 1: aguarda 2s + random(0,1)
Tentativa 2: aguarda 4s + random(0,1)
Tentativa 3: falha definitiva → exceção propagada
```

O jitter evita que múltiplos workers reintentem simultaneamente (thundering herd).

6.2 Dead Letter Queue

Mensagens recebidas 5 ou mais vezes sem processamento bem-sucedido são tratadas em dois níveis:

1. **DLQ AWS:** a fila SQS tem `RedrivePolicy` que move mensagens para `fila-analise-codigo-dlq` após 5 recebimentos
2. **DLQ local:** o worker registra um arquivo JSON em `testes/dlq/` com metadados da mensagem falha para auditoria

6.3 Estratégia de Fallback

Ponto crítico	Estratégia
Ollama indisponível	Retry com backoff. Se falhar, mensagem retorna à fila
SQS indisponível	Retry com backoff nas chamadas receive/delete
S3 indisponível	Log de warning, worker continua sem agregar
Worker encerra	Mensagem retorna à fila após VisibilityTimeout (120s)
Mensagem processada 5+ vezes	DLQ local + remoção da fila principal

7. Análise Experimental e Resultados

7.1 Configuração do Experimento

- **Infraestrutura:** 2 instâncias EC2 t3.small (1 vCPU, 2 GB RAM, CPU-only)
- **Modelo:** llama3.2:1b via Ollama (uma instância por EC2)
- **Storage compartilhado:** S3 bucket `tema7-pdp-resultados`
- **Corpus:** 4 arquivos Python do módulo `meu_codigo/`
- **Experimentos:** 1 worker (1 instância) vs 2 workers (2 instâncias separadas)

7.2 Resultados

Métrica	1 Worker	2 Workers
Tempo CPU acumulado	124.23s	111.17s
Tempo real de relógio	126.03s	56.97s
Throughput	0.032 arq/s	0.070 arq/s
Tokens totais	1590	1507
Workers utilizados	7c5b2961	7c5b2961, 5d921e3a
Conflitos detectados	0	0
Speedup	1x	2.21x

7.3 Análise dos Resultados

Speedup próximo do linear: com 2 workers em instâncias separadas, o tempo real caiu de 126.03s para 56.97s — um speedup de **2.21x**, muito próximo do ideal teórico de 2x. Isso confirma a natureza embaraçosamente paralela da tarefa.

Tempo real < Tempo CPU: no cenário de 2 workers, o tempo real (56.97s) é aproximadamente a metade do tempo CPU total (111.17s). Isso significa que os dois workers executaram majoritariamente em paralelo, com sobreposição quase total das operações de inferência.

Causa do speedup próximo do linear: cada instância EC2 tem seu próprio servidor Ollama e seu próprio vCPU. Não há contenção de recursos entre os workers — cada um processa seus arquivos de forma completamente independente, comunicando-se apenas via SQS (entrada) e S3 (saída).

Tokens estáveis: o consumo de tokens se manteve estável entre experimentos (~1.500 tokens totais), confirmando que o modelo processa o mesmo conteúdo independentemente de quantos workers são utilizados.

7.4 Eficiência do Paralelismo

```
Speedup ideal (teórico) = 2.00x
Speedup obtido    = 2.21x
Eficiência        = 2.21 / 2.00 = 110%
```

O speedup acima de 2x ocorre porque os arquivos não têm tamanho idêntico — o balanceamento dinâmico via SQS faz com que o worker mais rápido pegue o próximo arquivo assim que termina, evitando ociosidade.

8. Limitações e Propostas de Melhoria

8.1 Limitações Atuais

Limitação	Impacto
EC2 t3.small (1 vCPU, CPU-only)	Inferência lenta: 3–10 tokens/s

Limitação	Impacto
Modelo llama3.2:1b	Menor precisão comparado a modelos maiores
4 arquivos no corpus	Amostra pequena para análise estatística robusta
Sem GPU disponível	-5× mais lento que g4dn.xlarge com T4

8.2 Propostas de Melhoria

1. **GPU dedicada (g4dn.xlarge):** aceleraria inferência para ~50 tokens/s, viabilizando modelos maiores como llama3.2:3b ou Mistral 7B com mais qualidade nos testes gerados
2. **Mais workers e instâncias:** a arquitetura já suporta N workers — basta subir mais instâncias EC2 apontando para a mesma fila SQS. O speedup deve continuar próximo do linear até saturar o throughput do SQS
3. **DynamoDB para contagem distribuída:** substituir o `total_arquivos.txt` no S3 por um contador atômico no DynamoDB tornaria a detecção de conclusão mais precisa em cenários de alta concorrência
4. **Corpus maior:** testar com 20–50 arquivos permitiria medir o speedup com mais precisão estatística e demonstrar a escalabilidade do sistema

9. Conclusão

O sistema implementado demonstra com sucesso os conceitos de **paralelismo embaraçoso**, **comunicação distribuída via filas de mensagens** e **tolerância a falhas** aplicados à orquestração de um SLM. O resultado central — speedup de **2.21x** com 2 workers em instâncias EC2 separadas — confirma que a arquitetura distribui eficientemente a carga de inferência quando cada worker possui recursos de CPU dedicados.

A escolha do SLM auto-hospedado (Ollama + llama3.2:1b) permitiu controle total sobre a infraestrutura de inferência, evidenciando empiricamente que o bottleneck em ambientes single-core é o servidor de inferência compartilhado — e que a solução correta é distribuir os próprios servidores, não apenas os workers.

Apêndice A — Prompt v1: Geração de Testes

Arquivo: `prompts/v1 gerar_testes.txt`

Você é um Engenheiro de QA especialista em Python e PyTest. Analise o código fornecido e gere exclusivamente os testes unitários com pytest.

Regras estritas:

- Retorne APENAS o código Python pronto para execução, sem blocos markdown (sem ```).
- Não adicione nenhuma explicação, introdução ou texto antes/depois do código.
- Comece o arquivo diretamente com os imports necessários.
- Use exatamente o caminho de import que o usuário indicar na mensagem.
- Cubra casos normais, casos de borda e casos de erro.
- Use nomes de teste descritivos no formato `test_<funcao>_<cenario>`.

Apêndice B — Prompt v2: Detecção de Code Smells

Arquivo: [prompts/v2_detector_smells.txt](#)

Você é um Engenheiro de Software especialista em qualidade de código Python. Analise o código fornecido e identifique code smells e problemas de qualidade.

Retorne um JSON com a seguinte estrutura exata (sem texto adicional):

```
{
  "arquivo": "<nome do arquivo>",
  "smells": [
    {
      "tipo": "<nome do smell>",
      "linha": <numero da linha ou null>,
      "descricao": "<explicacao clara do problema>",
      "sugestao": "<como corrigir>"
    }
  ],
  "nota_qualidade": <numero de 0 a 10>,
  "resumo": "<frase curta com avaliacao geral>"
}
```

Tipos de smell a verificar: função muito longa, nome pouco descritivo, código duplicado, magic numbers, ausência de tratamento de erro, complexidade ciclomática alta, falta de validação de entrada.

Apêndice C — Relatório Final Consolidado (2 Workers)

```
{
  "total_arquivos_analisados": 4,
  "sucessos": 4,
  "falhas": 0,
  "tempo_total_acumulado_cpu_segundos": 111.17,
  "tempo_real_relogio_segundos": 56.97,
```



```
"throughput_arquivos_por_segundo": 0.070,  
"total_tokens_prompt": 918,  
"total_tokens_resposta": 589,  
"workers_utilizados": ["7c5b2961", "5d921e3a"],  
"modulos_processados": ["calculadora", "pedido", "string_utils", "usuarios"],  
"conflitos_detectados": []  
}
```